19. To Register Handlers, Configure Signal Actions, Handle Interrupts, Manage Signal Context, Verify Stability, Test Signal Events.

Code:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <string.h>

volatile sig_atomic_t signal_count = 0;

/* Signal handler */
void signalHandler(int sig, siginfo_t *info, void *context) {

    (void)context;

    signal_count++;

    printf("\n--- Signal Event ---\n");

    if (sig == SIGINT) {
        printf("Received SIGINT (Ctrl+C)\n");
    }
    else if (sig == SIGTERM) {
        printf("Received SIGTERM\n");
    }
    else if (sig == SIGUSR1) {
        printf("Received SIGUSR1\n");
    }

    if (info != NULL) {
        printf("Sender PID: %d\n",
               info->si_pid);
    }

    printf("Signal count: %d\n",
           signal_count);
}

/* Register signal handlers */
void registerHandlers() {

    struct sigaction action;

    memset(&action, 0, sizeof(action));

    /*
     * Use extended signal information.
     */
    action.sa_sigaction = signalHandler;

    /*
     * SA_SIGINFO allows the handler
     * to receive signal context.
     */
    action.sa_flags = SA_SIGINFO;

    sigemptyset(&action.sa_mask);

    /* Register SIGINT */
    if (sigaction(SIGINT, &action, NULL) == -1) {
        perror("sigaction SIGINT");
        exit(1);
    }

    /* Register SIGTERM */
    if (sigaction(SIGTERM, &action, NULL) == -1) {
```

```c
    /* Register SIGTERM */
    if (sigaction(SIGTERM, &action, NULL) == -1) {
        perror("sigaction SIGTERM");
        exit(1);
    }

    /* Register SIGUSR1 */
    if (sigaction(SIGUSR1, &action, NULL) == -1) {
        perror("sigaction SIGUSR1");
        exit(1);
    }

    printf("Signal handlers registered successfully.\n");
}

/* Display current signal mask */
void displaySignalMask() {

    sigset_t mask;

    if (sigprocmask(SIG_SETMASK, NULL, &mask) == -1) {
        perror("sigprocmask");
        return;
    }

    printf("\nSignal Mask Status:\n");

    printf("SIGINT  : %s\n",
           sigismember(&mask, SIGINT)
           ? "Blocked"
           : "Not Blocked");

    printf("SIGTERM : %s\n",
           sigismember(&mask, SIGTERM)
           ? "Blocked"
           : "Not Blocked");

    printf("SIGUSR1 : %s\n",
           sigismember(&mask, SIGUSR1)
           ? "Blocked"
           : "Not Blocked");
}

int main() {

    printf("Signal Handler Management System\n");
    printf("--------------------------------\n");

    printf("Process PID: %d\n",
           getpid());

    /* Register handlers */
    registerHandlers();

    /* Display signal mask */
    displaySignalMask();

    printf("\nSignal tests:\n");
    printf("1. Press Ctrl+C to generate SIGINT.\n");
    printf("2. The program will continue running.\n");
    printf("3. Press Ctrl+\\ only if you want to terminate it.\n\n");

    /*
     * Generate SIGUSR1 from the same process.
```

```c
    /*
     * Generate SIGUSR1 from the same process.
     */
    printf("Sending SIGUSR1...\n");

    if (kill(getpid(), SIGUSR1) == -1) {
        perror("kill");
    }

    printf("\nWaiting for signal events...\n");

    /*
     * Main loop remains stable even after
     * receiving handled signals.
     */
    for (int i = 1; i <= 10; i++) {

        printf("Program running... %d/10\n", i);

        sleep(1);
    }

    printf("\nProgram completed normally.\n");
    printf("Total signals handled: %d\n",
           signal_count);

    return 0;
}
```

Result:

```
sanjana3027@DESKTOP-QJF5AGU:        $kill19a.c
sanjana3027@DESKTOP-QJF5AGU:        <ill19a.c -o s19a
sanjana3027@DESKTOP-QJF5AGU:        a
Signal Handler Management System
--------------------------------
Process PID: 3362
Signal handlers registered successfully.

Signal Mask Status:
SIGINT  : Not Blocked
SIGTERM : Not Blocked
SIGUSR1 : Not Blocked

Signal tests:
1. Press Ctrl+C to generate SIGINT.
2. The program will continue running.
3. Press Ctrl+\ only if you want to terminate it.

Sending SIGUSR1...

--- Signal Event ---
Received SIGUSR1
Sender PID: 3362
Signal count: 1

Waiting for signal events...
Program running... 1/10
Program running... 2/10
Program running... 3/10
Program running... 4/10
Program running... 5/10
Program running... 6/10
Program running... 7/10
Program running... 8/10
Program running... 9/10
Program running... 10/10

Program completed normally.
Total signals handled: 1
sanjana3027@DESKTOP-QJF5AGU:
```

To Capture SIGINT, Forward Signals, Protect Shell Process, Terminate Foreground Jobs, Update Job States, Test Interrupt Handling.

Code:

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

pid_t foreground_pid = -1;

/* SIGINT handler for the shell */
void handleSIGINT(int sig) {

    (void)sig;

    printf("\nShell received SIGINT.\n");

    /*
     * If a foreground job exists,
     * forward SIGINT to that job.
     */
    if (foreground_pid > 0) {

        printf("Forwarding SIGINT to foreground job %d...\n",
               foreground_pid);

        kill(foreground_pid, SIGINT);
    }
    else {

        printf("Shell is protected from termination.\n");
        printf("No foreground job is running.\n");
    }
}

/* Create foreground job */
void runForegroundJob() {

    int status;

    foreground_pid = fork();

    if (foreground_pid < 0) {

        perror("fork");
        return;
    }

    /* Child process */
    if (foreground_pid == 0) {

        /*
         * Restore default SIGINT behavior
         * for the child.
         */
        signal(SIGINT, SIG_DFL);

        printf("Foreground job started.\n");
        printf("Child PID: %d\n",
               getpid());

        /*
         * Long-running foreground task.
         */
        for (int i = 1; i <= 20; i++) {
```

```c
        /*
         * Long-running foreground task.
         */
        for (int i = 1; i <= 20; i++) {

            printf("Foreground job running... %d/20\n",
                   i);

            sleep(1);
        }

        printf("Foreground job completed normally.\n");

        exit(0);
    }

    /* Parent waits for foreground job */
    printf("Foreground job started with PID %d.\n",
           foreground_pid);

    printf("Press Ctrl+C to interrupt the job.\n\n");

    waitpid(foreground_pid, &status, 0);

    /*
     * Check how the job ended.
     */
    if (WIFSIGNALED(status)) {

        printf("\nForeground job terminated by signal %d.\n",
               WTERMSIG(status));
    }
    else if (WIFEXITED(status)) {

        printf("\nForeground job exited normally.\n");
        printf("Exit status: %d\n",
               WEXITSTATUS(status));
    }

    /*
     * Job is no longer active.
     */
    foreground_pid = -1;
}

int main() {

    /*
     * Register SIGINT handler for shell.
     */
    signal(SIGINT, handleSIGINT);

    printf("SIGINT Forwarding Shell\n");
    printf("-----------------------\n");

    printf("Shell PID: %d\n",
           getpid());

    printf("Shell is protected from Ctrl+C.\n");

    printf("\nStarting foreground job...\n\n");

    runForegroundJob();
```

```c
    /*
     * Register SIGINT handler for shell.
     */
    signal(SIGINT, handleSIGINT);

    printf("SIGINT Forwarding Shell\n");
    printf("-----------------------\n");

    printf("Shell PID: %d\n",
           getpid());

    printf("Shell is protected from Ctrl+C.\n");

    printf("\nStarting foreground job...\n\n");

    runForegroundJob();

    printf("\nShell remains active after job termination.\n");
    printf("SIGINT handling test completed successfully.\n");

    return 0;
}
```

Result:

```
sanjana3027@DESKTOP-QJF5AGU:       $ nano Skill19b.c
sanjana3027@DESKTOP-QJF5AGU:       $ gcc Skill19b.c
sanjana3027@DESKTOP-QJF5AGU:       $ nano Skill19b.c
sanjana3027@DESKTOP-QJF5AGU:       $ gcc Skill19b.c -o s19b
sanjana3027@DESKTOP-QJF5AGU:       $ ./s19b
SIGINT Forwarding Shell
-----------------------
Shell PID: 3467
Shell is protected from Ctrl+C.

Starting foreground job...

Foreground job started with PID 3468.
Press Ctrl+C to interrupt the job.

Foreground job started.
Child PID: 3468
Foreground job running... 1/20
Foreground job running... 2/20
Foreground job running... 3/20
Foreground job running... 4/20
Foreground job running... 5/20
Foreground job running... 6/20
Foreground job running... 7/20
Foreground job running... 8/20
Foreground job running... 9/20
Foreground job running... 10/20
Foreground job running... 11/20
Foreground job running... 12/20
Foreground job running... 13/20
Foreground job running... 14/20
Foreground job running... 15/20
Foreground job running... 16/20
Foreground job running... 17/20
Foreground job running... 18/20
Foreground job running... 19/20
Foreground job running... 20/20
Foreground job completed normally.

Foreground job exited normally.
Exit status: 0

Shell remains active after job termination.
SIGINT handling test completed successfully.
sanjana3027@DESKTOP-QJF5AGU:       $
```